

SE3004 SOFTWARE TESTING

Muğla Sıtkı Koçman University

Software Engineering Department

# Term Project

## ShopEasy: A Comprehensive Test Suite

Applying Chapters 1–6 Testing Techniques to a Java E-Commerce Library

|                   |                                     |
|-------------------|-------------------------------------|
| <b>Due Date</b>   | 19 May 2026 / 23.59                 |
| <b>Submission</b> | GitHub repository link              |
| <b>Language</b>   | Java (JUnit 5, jqwik, Mockito)      |
| <b>Build Tool</b> | Maven                               |
| <b>Textbook</b>   | Effective Software Testing (Aniche) |

# 1. Overview

This project asks you to build a comprehensive, professional-grade test suite for ShopEasy, a pre-written Java library that simulates a small e-commerce system. You will apply one core testing technique from each of the first six chapters of Effective Software Testing (Aniche). Rather than writing production code, your primary deliverable is the test code itself — and the insight you demonstrate about why and how each technique works.

The project is designed so that all tasks operate on the same codebase. This means you will see how different techniques interact and complement each other, which is itself an important lesson in software quality engineering.

## 1.1 Task Summary

| Task | Technique                          | Chapter   | Weight |
|------|------------------------------------|-----------|--------|
| 1    | Specification-Based Testing        | Chapter 2 | 20 pts |
| 2    | Structural Testing & Code Coverage | Chapter 3 | 20 pts |
| 3    | Design by Contract                 | Chapter 4 | 15 pts |
| 4    | Property-Based Testing             | Chapter 5 | 20 pts |
| 5    | Mocks & Stubs                      | Chapter 6 | 20 pts |
| 6    | Reflection Report                  | Ch. 1–6   | 5 pts  |

# 2. System Under Test — ShopEasy

The starter project will be provided as a Maven project on GitHub. You must not modify the production classes (`src/main/java`) unless a task explicitly instructs you to add assertions or annotations. All your work goes into `src/test/java`.

## 2.1 Provided Classes

| Class / Interface                   | Responsibility                                                                                                                                            |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Product</b>                      | Holds product id, name, unit price, and stock quantity.                                                                                                   |
| <b>ShoppingCart</b>                 | Manages a list of cart items (product + quantity). Supports add/remove items, quantity update, and total price calculation. Applies percentage discounts. |
| <b>InventoryService (interface)</b> | Single method: <code>boolean isAvailable(Product p, int quantity)</code> . Used by <code>OrderProcessor</code> .                                          |
| <b>PaymentGateway (interface)</b>   | Single method: <code>boolean charge(String customerId, double amount)</code> . Used by <code>OrderProcessor</code> .                                      |
| <b>OrderProcessor</b>               | Orchestrates order creation: checks inventory, charges the customer, and returns an <code>Order</code> object on success.                                 |

|                        |                                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>PriceCalculator</b> | Stateless helper: computes final price given a base price, a discount rate (0–100), and a tax rate (0–100). |
| <b>Order</b>           | Immutable result object: orderId, customerId, items, total.                                                 |

## 2.2 Maven Dependencies (provided in pom.xml)

```

<dependency> <!-- JUnit 5 -->
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
<dependency> <!-- jqwik (property-based testing) -->
  <groupId>net.jqwik</groupId>
  <artifactId>jqwik</artifactId>
  <version>1.8.3</version>
  <scope>test</scope>
</dependency>
<dependency> <!-- Mockito -->
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>5.11.0</version>
  <scope>test</scope>
</dependency>
<plugin> <!-- JaCoCo for code coverage -->
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
</plugin>

```

### 3. Task Descriptions

#### Task 1: Specification-Based Testing · Chapter 2

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target class</b>  | PriceCalculator                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Minimum tests</b> | 10 meaningful test cases                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Technique</b>     | Domain testing: partition analysis + boundary value analysis                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Requirements</b>  | Identify all input dimensions (base price, discount rate, tax rate) and partition each into equivalence classes.<br>Identify and test boundary values between partitions for each dimension (on-point and off-point).<br>Cover exceptional inputs: zero price, zero discount, zero tax, maximum boundary values, and invalid inputs.<br>Use JUnit 5 <code>@ParameterizedTest</code> / <code>@CsvSource</code> wherever multiple test cases share the same structure.<br>Write a brief comment above each test method explaining which partition or boundary it covers. |
| <b>Deliverable</b>   | PriceCalculatorSpecTest.java                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

#### Task 2: Structural Testing & Code Coverage · Chapter 3

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target class</b>  | ShoppingCart                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Minimum tests</b> | Achieve $\geq 80\%$ branch coverage on ShoppingCart                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Technique</b>     | Structural testing guided by JaCoCo branch coverage report                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Requirements</b>  | Write an initial test suite based on the specification, then run: <code>mvn jacoco:report</code><br>Open <code>target/site/jacoco/index.html</code> and identify uncovered branches.<br>Add tests specifically to cover the missing branches until you reach $\geq 80\%$ branch coverage.<br>Include a screenshot or copy of the final JaCoCo summary table in your report.<br>Bonus (+5 pts): Use PIT mutation testing ( <code>mvn org.pitest:pitest-maven:mutationCoverage</code> ) and analyse at least 2 surviving mutants — explain why each survived and how you would kill it. |
| <b>Deliverable</b>   | ShoppingCartStructuralTest.java + JaCoCo screenshot in report                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

#### Task 3: Design by Contract · Chapter 4

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target classes</b> | ShoppingCart, PriceCalculator, OrderProcessor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Technique</b>      | Pre-conditions, post-conditions, and invariants using Java assert                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Requirements</b>   | Add assert statements to ShoppingCart and PriceCalculator to enforce the contracts listed below.<br>Write a separate ContractTest.java that verifies (a) contracts hold for valid inputs and (b) contracts are violated (AssertionError) for invalid inputs. Run tests with <code>-ea</code> (assertions enabled) in your IDE or Maven Surefire config.<br>Contracts to implement:<br>ShoppingCart.addItem(product, qty): pre — <code>product != null, qty &gt; 0</code> ; post — <code>cart size increased by one</code> .<br>ShoppingCart.applyDiscount(rate): pre — <code>0 &lt;= rate &lt;= 100</code> ; post — <code>total() &lt; previous total (if rate &gt; 0)</code> .<br>PriceCalculator.calculate(base, discount, tax): pre — <code>base &gt;= 0, 0 &lt;= discount &lt;= 100, 0 &lt;= tax &lt;= 100</code> ; post — <code>result &gt;= 0</code> .<br>ShoppingCart invariant: <code>total()</code> is always $\geq 0$ after any operation.<br>Document each contract in a short Javadoc comment on the method. |
| <b>Deliverable</b>    | Modified ShoppingCart.java & PriceCalculator.java + ContractTest.java                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

#### Task 4: Property-Based Testing · Chapter 5

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target classes</b> | PriceCalculator, ShoppingCart                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Minimum</b>        | 3 distinct properties with jqwik @Property methods                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Technique</b>      | Property-based testing with jqwik                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Requirements</b>   | <p>Define and test at least 3 properties. You must use at least one custom @Provide method. Suggested properties (you may use these or design your own — original properties earn full credit):</p> <ul style="list-style-type: none"><li>Monotonicity: For any fixed base and tax, increasing the discount rate never increases the final price.</li><li>Identity: A 0% discount and 0% tax returns exactly the base price.</li><li>Boundedness: The final price is always <math>\geq 0</math> and <math>\leq \text{base} * (1 + \text{maxTaxRate} / 100)</math>.</li><li>Cart commutativity: Adding item A then item B to a cart yields the same total as adding B then A.</li></ul> <p>For each property, briefly explain in a comment: (a) what the property means, (b) what class of bugs it would catch.</p> <p>If jqwik finds a failing case, do not just fix the test — investigate and explain the root cause in your report.</p> |
| <b>Deliverable</b>    | ShopEasyPropertyTest.java                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

#### Task 5: Mocks & Stubs · Chapter 6

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target class</b> | OrderProcessor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Minimum</b>      | 4 distinct test scenarios using Mockito mocks                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Technique</b>    | Test doubles: mocking InventoryService and PaymentGateway                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Requirements</b> | <p>Use @Mock and @InjectMocks (Mockito + JUnit 5 extension) to isolate OrderProcessor from its dependencies.</p> <p>Cover all of the following scenarios:</p> <ul style="list-style-type: none"><li>Happy path: inventory available, payment succeeds → Order object returned.</li><li>Inventory failure: isAvailable() returns false → no payment attempted, method returns null or throws.</li><li>Payment failure: isAvailable() returns true, charge() returns false → Order not created.</li><li>Partial quantity: only some units are available — define and test the expected behavior.</li></ul> <p>Use verify() to assert that charge() is never called when inventory is unavailable.</p> <p>In your report, discuss the trade-off: what does mocking allow you to test that you could not test otherwise, and what does it prevent you from testing?</p> |
| <b>Deliverable</b>  | OrderProcessorMockTest.java                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

#### Task 6: Reflection Report · Chapter 1–6

|                            |                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Format</b>              | PDF, 1–2 pages (plain prose, no bullet-only answers)                                                                                                                                                                                                                                                                                                                           |
| <b>Questions to answer</b> | <p>Which bugs (if any) did each technique reveal in the starter code? Give specific examples.</p> <p>Which technique do you think was most effective per unit of effort? Justify your answer.</p> <p>Based on the testing pyramid (Chapter 1), how would you classify your test suite? What is missing?</p> <p>If you had one more week, what would you test next and why?</p> |

## 4. Submission Instructions

Submit a single GitHub repository link via the course portal by the deadline. Your repository must contain the following structure:

```
shopeasy-tests/
├── pom.xml
├── src/
│   ├── main/java/shopeasy/      ← provided (do not modify, except Task 3 classes)
│   └── test/java/shopeasy/
│       ├── PriceCalculatorSpecTest.java      (Task 1)
│       ├── ShoppingCartStructuralTest.java   (Task 2)
│       ├── ContractTest.java                 (Task 3)
│       ├── ShopEasyPropertyTest.java         (Task 4)
│       └── OrderProcessorMockTest.java       (Task 5)
└── report/
    ├── reflection.pdf                 (Task 6)
    └── jacoco-screenshot.png          (Task 2)
```

Additional requirements:

- All tests must pass with `mvn test` (run this before submitting).
- Do not commit IDE-specific files (`.idea/`, `*.iml`). Use the provided `.gitignore`.
- Commit regularly — at least one commit per task. Your Git history is part of the evaluation.
- You may work individually or in pairs. If working in pairs, both names must appear in an `AUTHORS.md` file at the root.

## 6. Academic Integrity

All submitted work must be your own (or your pair's). You may use the course textbook, lecture notes, official library documentation, and Stack Overflow for reference. You may not copy test code from other students or from AI-generated output without understanding and being able to explain every line. Plagiarism will result in a grade of zero for the entire project.

Discussing high-level ideas and design decisions with classmates is encouraged. Sharing actual test code is not.

## 7. Tips for Success

- Read the starter code carefully before writing a single test. Understand what each method is supposed to do.
- Do the tasks in order — the specification-based tests you write in Task 1 will give you a head start on structural coverage in Task 2.
- For Task 4, think about what invariants should always hold regardless of the input, not just for specific values.
- For Task 5, draw a dependency diagram of `OrderProcessor` before writing any tests — it clarifies what needs to be mocked.
- Run `mvn test` frequently. Catching a broken test early is much less painful than debugging at the end.
- Start the reflection report early and update it as you go — it is much easier to record observations in the moment.

---

*Good luck — and remember: a test that finds a bug is a success, not a failure.*